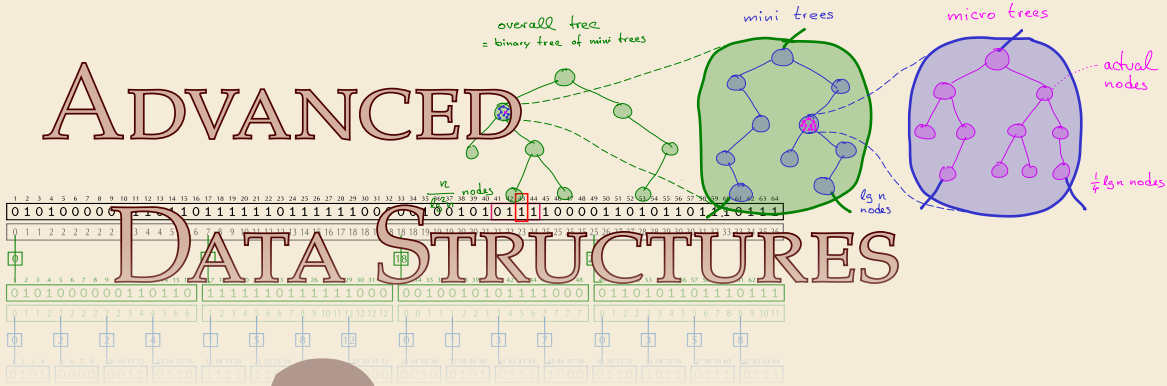


Advanced Data Structures

2 Adaptive Trees

Advanced Data Structures · Summer 2026

Prof. Dr. Sebastian Wild

CS650 (Summer 2026)
Philipps-Universität Marburg
version `2026-04-28 12:18 H`

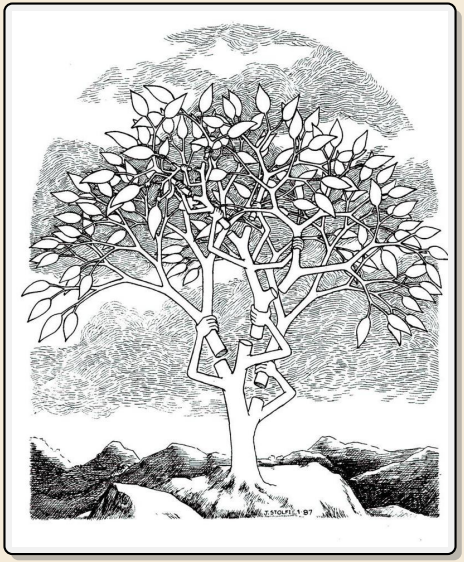
Outline

2 Adaptive Trees

2.1 Move-to-Root Heuristic
2.2 Move-to-Root Analysis
2.3 Splay Trees
2.4 Analysis of Splay Trees
2.5 Biased Search Trees
2.6 Dynamic Optimality
2.7 Lazy Search Trees

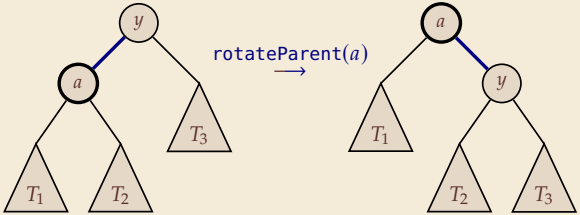
Self-Adjusting Data Structures

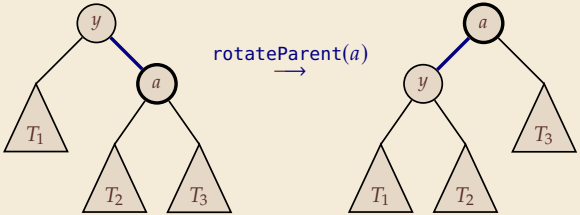
Idea: set of rules that makes data structure automatically adapt to use case



2.1 Move-to-Root Heuristic

Rotations


y
a
T1
T2
T3
rotateParent(a)
a
y
T1
T2
T3


y
a
T1
T2
T3
rotateParent(a)
a
y
T1
T2
T3

Move-to-root

Upon an access to an element, `rotateParent` until accessed element is at root!

Allen, Munro: *Self-Organizating Binary Search Trees*, JACM **1978**

Inspired by **Move-To-Front heuristic** of self-adjusting linked **lists**

- cost via MTF is at most 2 times cost of optimal **static ordering**
- MTF is 2-competitive as an online algorithm
  (at most 2 times the cost of *any* algorithm)
- (both true under "basic cost model" only)
- with resource augmentation, LRU in paging very successful

Maybe, Move-to-root is similarly successful!

2.2 Move-to-Root Analysis

Iid Accesses

Iid Model: Suppose the accesses to keys $[1..n]$ in BST are *i.i.d. randomly* drawn with probabilities $p_1, \ldots, p_n$.

$\rightsquigarrow$ Expected search cost in a BST T given by $\sum_{i=1}^{n} p_i \cdot \mathrm{depth}_T(i)$

- If p_i known up front, can compute **static optimal BST** via dynamic programming

$\rightsquigarrow$ Cost of this optimal tree C_{OPT} can be bounded in terms of the entropy

Lemma 2.1 (Bayer 1975)

$$\mathcal{H} - \lg \mathcal{H} - \lg e + 1 \;\leq\; C_{OPT} \;\leq\; \mathcal{H} + 1 \quad \text{for} \quad \mathcal{H} = \sum_{i=1}^{n} p_i \lg(1/p_i)$$

Bayer: *Improved Bounds on the Costs of Optimal and Balanced Binary Search Trees*, M.Sc. Thesis, MIT **1975**

Note: C_{OPT} is for successful search / depth of internal nodes
The case of accesses to leaves only, with access to $\boxed{j}$ w/p q_j, $j = 0, \ldots, n$ even cleaner:

$$\mathcal{H} \;\leq\; \sum_{j=0}^{n} q_j \cdot (1 + \mathrm{depth}_T(\boxed{j})) \;\leq\; \mathcal{H} + 2$$

Move-to-Root on iid Accesses

Lemma 2.2 (MTR ancestors)

Let T be a tree maintained by the move-to-root heuristic and $i < j$ two keys.
Then i is an **ancestor** of j iff the most recent request for i came after the most recent requests of any of $i+1, \ldots, j$.
Similarly, j ancestor of i iff j requested more recently than any of $i, \ldots, j-1$.

Recall $A^i_j = [i \text{ ancestor of } j]$

Corollary 2.3

Under the iid accesses model and MTR heuristic, $\mathbb{P}[A^i_j] = \begin{cases} \dfrac{p_i}{p_i + \cdots + p_j} & i < j \\ \dfrac{p_i}{p_j + \cdots + p_i} & i > j \end{cases}$

Move-to-Root on iid Accesses – Analysis

Theorem 2.4 (MTR cost)

Let C_{MTR} be the expected search cost under the i.i.d. model in a tree T maintained by MTR.

$$C_{MTR} = 1 + 2 \sum_{1 \le i < j \le n} \frac{p_i p_j}{p_i + \cdots + p_j} \le 1 + 2 \ln 2 \cdot \mathcal{H}$$

Move-to-Root has close to optimal search cost in the i.i.d. model!

Proof:

$$C_{MTR} = \sum_{i=1}^{n} p_i \cdot (1 + \text{depth}(i)) = \sum_{i=1}^{n} p_i \cdot \left(1 + \sum_{j \ne i} \mathbb{E}[A_i^j]\right)$$

$$= 1 + \sum_{i=1}^{n} \sum_{j \ne i} p_i \mathbb{E}[A_i^j]$$

$$= 1 + 2 \sum_{1 \le i < j \le n} \frac{p_i p_j}{p_i + \cdots + p_j}$$

This proves the first part.

Move-to-Root on iid Accesses – Analysis

Proof (cont.):

Mehlhorn's trick: $\delta_{i,j} := \dfrac{p_i + \cdots + p_j}{p_i}$ $\quad\rightsquigarrow\quad 1 = \delta_{i,i} \le \delta_{i,i+1} \le \cdots \le \delta_{i,n} \le \dfrac{1}{p_i}$

$$
\begin{aligned}
C_{MTR} &= 1 + 2\sum_{i=1}^{n-1} p_i \sum_{j=i+1}^{n} \frac{p_j}{p_i + \cdots + p_j} \\
&= 1 + 2\sum_{i=1}^{n-1} p_i \sum_{j=i+1}^{n} \frac{\delta_{i,j} - \delta_{i,j-1}}{\delta_{i,j}} \\
&< 1 + 2\sum_{i=1}^{n-1} p_i \sum_{j=i+1}^{n} \int_{x=\delta_{i,j-1}}^{\delta_{i,j}} \frac{1}{x}\,dx \\
&= 1 + 2\sum_{i=1}^{n-1} p_i \int_{x=\delta_{i,i}}^{\delta_{i,n}} \frac{1}{x}\,dx \\
&= 1 + 2\sum_{i=1}^{n-1} p_i \ln(\delta_{i,n}/\delta_{i,i}) \le 1 + 2\ln 2 \sum_{i=1}^{n-1} p_i \lg(1/p_i) = 1 + 2\ln 2 \cdot \mathcal{H}
\end{aligned}
$$

$$\boxed{a \le b \implies (b-a)\cdot\tfrac{1}{b} = \int_a^b \tfrac{1}{b}\,dx < \int_a^b \tfrac{1}{x}\,dx}$$

$$\boxed{\int_{x=a}^{b} \tfrac{1}{x}\,dx = \ln(b) - \ln(a) = \ln(b/a)}$$

■

Unbalanced BSTs under i.i.d. model

Interestingly, the same cost result from unbalanced BSTs!

- Suppose we repeatedly draw x i.i.d. from $[1..n]$ w/p $p_1, \ldots, p_n$
- We insert x into initially empty unbalanced BST T
- **If x is already contained in T, the insertion has no effect**
- Repeat until saturation (i.e., until we have seen every value $x \in [1..n]$)

Since insertions are i.i.d., probability that i is inserted first among $[i..j]$ is $\dfrac{p_i}{p_i + \cdots + p_j}$

$$\rightsquigarrow \mathbb{P}[A^i_j] = \begin{cases} \dfrac{p_i}{p_i + \cdots + p_j} & i < j \\ \dfrac{p_i}{p_j + \cdots + p_i} & i > j \end{cases}$$

$$\rightsquigarrow C_T = 1 + 2 \sum_{1 \le i < j \le n} \frac{p_i p_j}{p_i + \cdots + p_j} \le 1 + 2 \ln 2 \cdot \mathcal{H}$$

Both unbalanced BST and Move-to-Root behave well under i.i.d. model.

Move-to-Root's Blind Spot

Problem: *Can't rely on accesses to be random.*

Theorem 2.5 (Move-to-Root Bad Case)

There exists a sequence of n requests, such that starting from any tree T, the amortized access cost over the course of the n requests is $\Omega(n)$.

Proof:
see exercises

2.3 Splay Trees

Splay Trees

Splay trees by now the most widely known self-adjusting data structure.

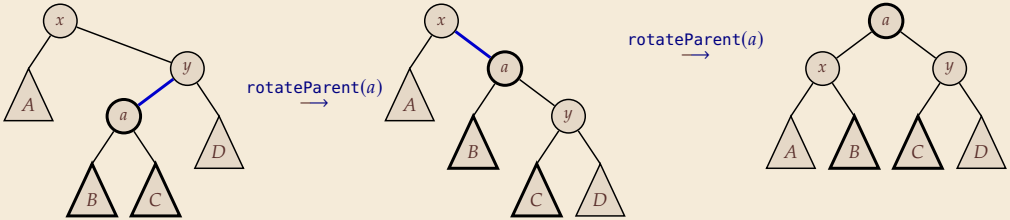
- 👍 reasonably simple and fast
- 👍 simple to implement
- 👍 remarkable theoretical properties (*access theorems*) ⇝ later
- 👎 many writes and pointer chases

Sleater, Tarjan: *Self-Adjusting Binary Search Trees*, JACM **1985**

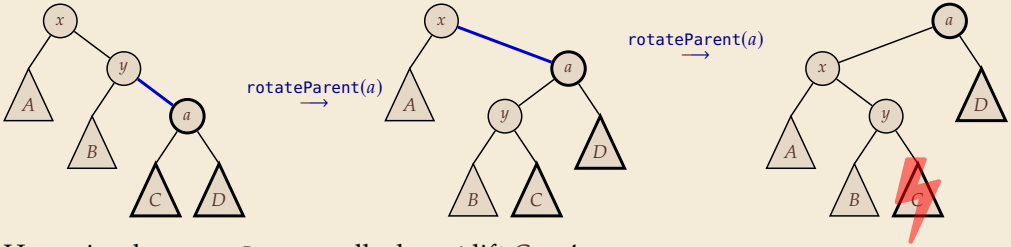
Move-to-root reloaded

What's wrong in Move-to-front?

Let's look at the possible cases of parent **and grandparent**



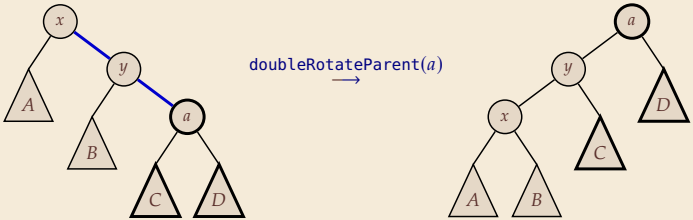
Using simple `rotateParent` calls, the subtree we came from (B or C) is lifted up!



Here, simple `rotateParent` calls do **not** lift C up!

The Fix

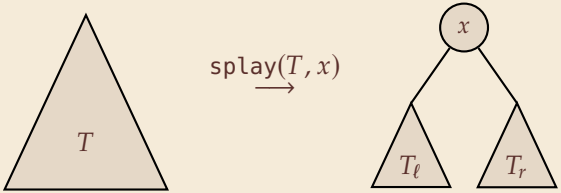
In the case that both parents are right children, do a double rotation



Now both C and D are lifted up!

Splay

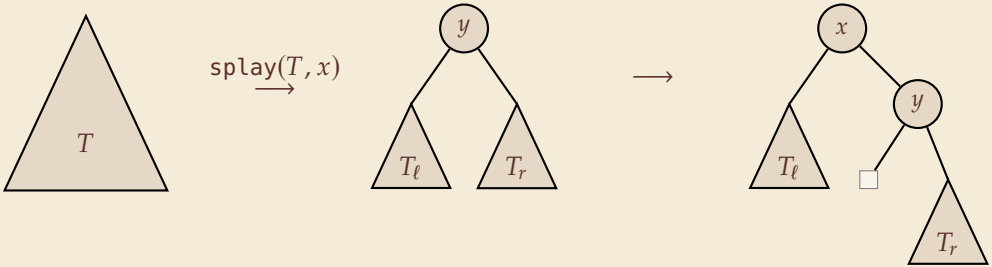
Aggressive restructuring allows to base all operations on single primitive: `splay`




```
def splay(T, x):
    while x ≠ T.root
        p = parent(x)
        if p == T.root
            rotateParent(x)  # Zig Case
        else:
            g = parent(p)
            if p == g.left ∧ x == p.left or p == g.right ∧ x == p.right:
                doubleRotateParent(x)  # Zig-Zig Case
            else:
                rotateParent(x)  # Zig-Zag Case
                rotateParent(x)
```

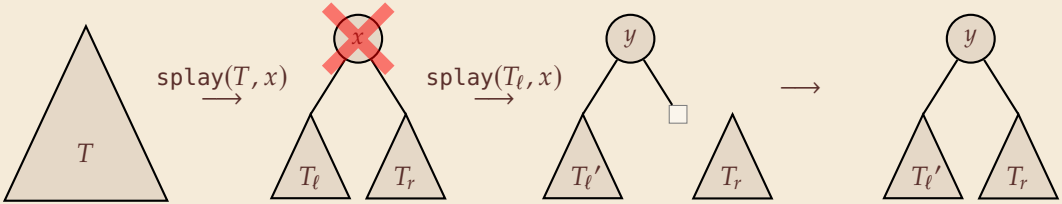
Splay – Insert

1. $\texttt{splay}(x, T) \rightsquigarrow$ root y and $T_\ell \leq y \leq T_r$ (key order)
2. If $x = y$ (key already stored) return.
3. Otherwise, w.l.o.g. $x < y$, can simply make x new root.



Splay – Delete

1. `splay`(x, T) $\rightsquigarrow$ root x (or not part of tree and we're done)
2. Remove x, left with T_ℓ and T_r
3. `splay`(x, T_ℓ) $\rightsquigarrow$ root $y = \texttt{max}(T_\ell)$ and $y.right = \texttt{null}$
4. $y.right := T_r$



Note: We've effectively built **`join`** $\rightsquigarrow$ naturally supported via `splay`.

`split` even more directly supported via `splay`: `split`(T, x): `splay`(T, x), return $(x.left, x)$

2.4 Analysis of Splay Trees

Rotation Cost

Note: In all operations, we follow a search path and splay.
(For delete twice)

splay walks back up search path

- $\rightsquigarrow$ use $k = \Theta(d)$ rotations for splaying node at depth d.
- $\rightsquigarrow$ cost is dominated by $\boldsymbol{k}$ **= #rotations in `splay`** calls.

Clearly, individual operations could be expensive (use $k = \Omega(n)$ rotations).

$\rightsquigarrow$ Can only hope for an ***amortized*** bound on k

We define a potential function $\Phi = \Phi(T)$ such that

$\Delta\Phi$ = change in potential

Goal: If `splay`(x) uses k rotations, then $\boxed{k + \Phi' - \Phi \quad \leq \quad 1 + 3 \cdot \big(r'(x) - r(x)\big)}$

change in "rank" of x

$\rightsquigarrow$ Released potential can pay for actual cost k

Splay Tree Potential

We choose a general method that will give several cool results.

- For that, we allow each node x to have a ***weight*** $w(x)$
- The ***size*** of a node x is defined as $\boxed{s(x) \;:=\; \sum_{\substack{v \in T: \\ x \text{ ancestor of } v}} w(v)}$
- The ***rank*** of node x is $\boxed{r(x) \;:=\; \lg\big(s(x)\big)}$
- The ***potential*** of a tree is $\boxed{\Phi \;=\; \sum_{v \in T} r(v)}$
- We always denote with Φ, r, s the quantities **before** an operation and with Φ', r', s' the quantities **after** an operation

The Access Lemma

```
def splay(T, x):
    while x ≠ T.root
        p = parent(x)
        if p == T.root
            rotateParent(x)  # Zig Case
        else:
            g = parent(p)
            if p == g.left ∧ x == p.left or p == g.right ∧ x == p.right:
                doubleRotateParent(x)  # Zig-Zig Case
            else:
                rotateParent(x)  # Zig-Zag Case
                rotateParent(x)
```

$$\boxed{\Phi = \sum_{v \in T} r(v)}$$

$$\boxed{r(x) := \lg\big(s(x)\big)}$$

$$\boxed{s(x) := \sum_{\substack{v \in T: \\ x \text{ ancestor of } v}} w(v)}$$

Lemma 2.6 (Access Lemma)

For **any** weakly positive node weights w, the amortized cost

(a) for a **Zig** is $\leq 1 + 3\big(r'(x) - r(x)\big)$

(b) for a **Zig-Zig** is $\leq 3\big(r'(x) - r(x)\big)$

(c) for a **Zig-Zag** is $\leq 3\big(r'(x) - r(x)\big)$

Access Lemma – Proof

Splay Cost

Corollary 2.7

The amortized cost to splay a tree with root t at node x is at most

$$3\big(r(t) - r(x)\big) + 1 \;=\; O\left(\log\left(\frac{s(t)}{s(x)}\right)\right)$$

Proof:

Splay does rotations involving the same node x until it is the root.

- ⇝ The rank differences telescope.
- ⇝ Result follow from Lemma 2.6.

Splay Tree – Results

Theorem 2.8 (Balance Theorem)

Consider a splay tree containing n keys and an arbitrary sequence of m accesses to these keys. The total access time is $O(n \log n + m \log n)$. ◀

Splay Tree – Results [2]

Theorem 2.9 (Static Optimality)

Consider a splay tree containing n keys m accesses in the i.i.d. model where every key is accessed at least once.
The total access time is $O(m(\mathcal{H} + 1))$. ◀

Splay Tree – Results [3]

Theorem 2.10 (Static Finger Theorem)

For any fixed node f (the "finger"), the amortized cost of accessing x is $O(\log(|x - f|))$. ◀

Here $|x - y|$ in the splay tree T means the *rank distance*, i. e., the number of nodes $z \in T$ with $y \leq z \leq x$ (or $x \leq z \leq y$)

Splay Tree – Results [4]

Theorem 2.11 (Working Set Theorem)

The amortized cost of accessing node x is $O(\log D)$, where D is the number of distinct items accessed since the last time x was accessed. (For the first access to x, we set $D = n$.) ◀

2.5 Biased Search Trees

2.6 Dynamic Optimality

The BST Model

2.7 Lazy Search Trees